Artifact for EMSOFT 2026: Suborbital and Orbital Real-Time Localization of Gamma-Ray Bursts via Utility-Guided Coordination

This repository contains the source code, processed datasets, and analysis tools supporting the EMSOFT 2026 paper, *"Suborbital and Orbital Real-Time Localization of Gamma-Ray Bursts via Utility-Guided Coordination."*

Repository: https://github.com/Daisy0419/GRB-Search-Pipeline.git

Artifact dataset: https://zenodo.org/records/21497891 DOI 10.5281/zenodo.21497891

Artifact documentation: `INSTALL`, `REQUIREMENTS`, `STATUS`, and `LICENSE`. The paper is included as `paper.pdf`.

Table of Contents

System Requirements

> **Architecture note:** Sections 1-3 support the complete artifact workflow on Linux ARM64 (`aarch64`). Section 4 additionally provides a Linux x86-64 environment for reproducing the Intel measurements in Figure 4.

- OS: Linux
- Supported architectures:
    - ARM64 (`aarch64`) for the complete training and evaluation workflow and the Jetson benchmarks
    - Intel/AMD x86-64 for the optional Intel benchmarks in Section 4
- Required command-line tools: Git, CMake, Make, a C++17 compiler with OpenMP support, `wget`, `curl`, and `tar`
- Reference platform used in the paper: NVIDIA Jetson Orin NX
    - 8 ARM Cortex-A78AE v8.2 CPU cores
    - CPU frequency fixed at 1.5 GHz
    - 16 GB DRAM
    - GPU not used
- Reference Intel platform used for Figure 4: 2.3 GHz Intel Xeon Gold 5118 with 256 GB DRAM
- Required storage after extraction: **at least 65 GB**, plus temporary space for the downloaded archive
    - Repository: 200 MB
    - Miniconda: 800 MB
    - Conda environment: 2.8 GB
    - LEMON: 70 MB
    - Transients and models: approximately 30 GB
    - Generated maps: approximately 30 GB

The provided `cosipy-312-deps.yaml` contains packages exported from the ARM64 reference platform and is used in Sections 1-3. The separate `cosipy-312-intel.yml` file is used only for the Intel benchmark workflow in Section 4.

The paper's timing results are **platform-sensitive**. Direct timing comparisons should use the corresponding reference platform: the Intel Xeon platform or Jetson Orin NX for Figure 4, and the Jetson Orin NX for the remaining timing experiments.

Overview

This artifact contains the software and data for offline experiments that simulate and evaluate the proposed on-board observation workflow. It does not acquire data from a live instrument or control a physical telescope.

Proposed On-Board Workflow

During a real observation, the gamma-ray telescope would receive a mixture of source and background events without knowing which events came from the transient. The proposed workflow would:

1. **Gather events** after detecting a transient.
2. **Choose when to generate a map** using the observed events, estimated background rate, overall deadline, and pre-trained utility tables.
3. **Generate a likelihood map** representing the probable transient location.
4. **Plan the optical search** by converting the map into telescope-FoV tiles and running the GCP planning algorithm.
5. **Search for the transient** until it is detected or the deadline expires.

In the artifact, the event stream and optical search are simulated. The likelihood-mapping, utility-estimation, tiling, and GCP planning code is executed normally. Telescope movement and observation are simulated using the slew- and dwell-time models.

Artifact Experiment Workflow

The artifact experiments consist of offline training followed by evaluation.

Phase A: Training

1. **Generate training maps.** Use all 52,800 simulated training transients in each of the short- and long-transient scenarios. The long-transient scenario is named `longlow` in directory and result filenames. Generate each map using the true transient length and record the source-event count, background-event count, and map-generation time.
2. **Run search planning.** For each training map and telescope FoV, run GCP with different search budgets. Record the planning time and the minimum modeled slew-and-dwell time required to reach a tile containing the true source.

The utility query engine will use these measurements to estimate mapping time, planning time, and detection probability as functions of the observed event counts and remaining deadline. The paper uses `20 x 20` source/background bins and confidence level `q = 0.95`.

Phase B: Evaluation

Evaluation uses 10,000 test transients from each scenario. Use `-s 10000 -r 1957` in every map-generation run so that all policies use the same test transients.

1. **Generate utility-guided test maps.** Generate maps for both telescope FoVs at every evaluated deadline because the utility policy depends on the FoV-specific training table:
    - short: `10, 15, 20, 25, 30, 35, 40, 45, 50` seconds;
    - longlow: `30, 40, 50, 60, 70, 80, 90, 100, 110` seconds; and

- FoVs: `2.5 x 2.5` and `5.36 x 4.5`.

Save both the generated maps and the mapping-statistics CSV for every scenario, FoV, and deadline.

2. **Generate deadline-oblivious test maps.** Run the `nodeadline` mapping policy once for the short scenario and once for the longlow scenario. These maps do not depend on the FoV or deadline and can therefore be reused in all corresponding GCP evaluation runs.

   The `nodeadline` policy ignores the deadline when choosing when to begin mapping. It is still evaluated under each finite end-to-end deadline in the next step.

3. **Run GCP and simulate the search.** For both scenarios and both FoVs, run `sp_verify` at every evaluated deadline:
   - use the matching FoV- and deadline-specific maps for the utility policy; and
   - reuse the same `nodeadline` maps at every deadline.

   The evaluation accounts for the event-gathering time, map-generation time, GCP planning time, and modeled telescope slew-and-dwell time. It records whether the simulated search reaches a tile containing the true source before the overall deadline.

4. **Aggregate the results.** Calculate the mapping error, processing times, and detection probability for each combination of scenario, FoV, policy, and deadline.

Directory Structure

The artifact uses the source repository and downloaded transient-data directory shown below. Generated and precomputed results are stored inside the source repository.

```
~/
|-- GRB-Search-Pipeline/             # Source code and experiment results
|   |-- cosipy/                      # Python likelihood-map generation
|   |   |-- pyproject.toml
|   |   `-- cosipy/
|   |       `-- ts_map/              # Map-generation and utility code
|   |-- search-planning/             # C++ GCP search-planning code
|   |   |-- include/                 # C++ header files
|   |   |-- src/                     # C++ source files
|   |   |-- tilings/                 # FoV tilings and source-tile tables
|   |   `-- build/                   # sp_train and sp_verify executables
|   |-- results/                     # Newly generated outputs and figures
|   |   |-- run_training.py          # Runs GCP on training maps
|   |   `-- run_validation.py        # Evaluates generated test maps
|   |-- precomputed_results/         # Distributed results and visualization notebook
|   |-- cosipy-312-deps.yaml         # Python environment configuration
|   |-- cosipy-312-intel.yml         # Intel benchmark environment
|   |-- INSTALL.md                   # Brief installation and smoke-test instructions
|   |-- LICENSE.md                   # License placeholder
|   |-- README.md                    # Complete artifact and reproduction instructions
|   |-- REQUIREMENTS.md              # Hardware and software requirements
|   |-- STATUS.md                    # Requested badges and artifact scope
|   `-- paper.pdf                    # Latest paper version
`-- transients/                      # Downloaded models and transient datasets
    |-- models/
```

```
    |-- emsoft_training_short/
    |-- emsoft_training_longlow/
    |-- emsoft_test_short/
    |-- emsoft_test_longlow/
    `-- dc3_benchmark_data/
```

The large transient datasets under `~/transients/` are downloaded separately and are not stored in the Git repository. Newly generated files are written under `~/GRB-Search-Pipeline/results/`, while the distributed files under `~/GRB-Search-Pipeline/precomputed_results/` should remain unchanged.

1 Environment Setup and Data Download

1.1 Local Installation

1.1.1 Clone Repository

Clone the repository to the path of your choice. All commands listed hereafter assume it is placed directly into your home directory.

Clone the main repository:

```
cd ~
git clone https://github.com/Daisy0419/GRB-Search-Pipeline.git
```

Clone the map generation code:

```
cd ~/GRB-Search-Pipeline
git clone -b tsmap-artifact https://github.com/McKelvey-Engineering-CSE/cosipy.git
```

1.1.2 Python Environment Setup

> **Architecture requirement:** This subsection configures the ARM64 (`aarch64`) environment used for Sections 1-3. For the optional Intel/x86-64 benchmark, use the separate environment instructions in Section 4.1.

Verify the machine architecture:

```
uname -m
```

The output must be:

```
aarch64
```

Stop the installation if the machine does not use the supported architecture:

```
if [ "$(uname -m)" != "aarch64" ]; then
    echo "ERROR: Sections 1-3 require Linux aarch64." >&2
    exit 1
fi
```

We recommend using Conda to configure the Python environment.

If Conda is not already installed, download and install the Linux `aarch64` version of Miniconda. Change `~/conda` if you prefer another installation location.

```
cd ~/GRB-Search-Pipeline

export CONDA_DIR="${HOME}/conda"

wget -q \
    https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-aarch64.sh \
    -O miniconda.sh

bash miniconda.sh -b -p "${CONDA_DIR}"
rm miniconda.sh
```

Make Conda available in the current shell:

```
. "${CONDA_DIR}/etc/profile.d/conda.sh"
conda config --system --set channel_priority flexible
```

Create the Python environment using the provided ARM64-specific YAML file:

```
cd ~/GRB-Search-Pipeline
conda env create -f cosipy-312-deps.yaml
```

Activate the `cosipy-312` environment:

```
conda activate cosipy-312
```

Install the artifact branch of COSIpy:

```
cd ~/GRB-Search-Pipeline/cosipy
git switch tsmap-artifact

python -m pip install "poetry-core>=2,<3"
python -m pip install --no-deps -e .
```

Install Jupyter Notebook for result visualization:

```
python -m pip install notebook
```

1.1.3 C++ Environment Setup

(1) LEMON Graph Library (Required)

The Lemon Graph Library is used to compute the minimum-weight perfect matching used in our Greedy Christofides Pathfinding algorithm.

1. Download, extract, and build the LEMON Graph Library to the directory of your choice. All commands listed hereafter assume it is placed directly in your home directory.

```
wget http://lemon.cs.elte.hu/pub/sources/lemon-1.3.1.tar.gz
tar xvfz lemon-1.3.1.tar.gz
cd lemon-1.3.1
mkdir build && cd build
cmake ..
make -j
```

2. Set the necessary environment variables in your shell (change `~/lemon-1.3.1` to your preferred location).

```
export LEMON_SOURCE_DIR=~/lemon-1.3.1
export LEMON_BUILD_DIR=~/lemon-1.3.1/build
```

(2) HDF5 Library (Required)

The search-planning code reads likelihood maps stored in HDF5 files. Install HDF5 under your home directory so that administrator privileges are not required.

Download, build, and install HDF5:

```
cd ~
git clone --depth 1 https://github.com/HDFGroup/hdf5.git hdf5-source

cmake -S ~/hdf5-source -B ~/hdf5-build \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_INSTALL_PREFIX="${HOME}/hdf5" \
    -DCMAKE_INSTALL_LIBDIR=lib \
    -DBUILD_SHARED_LIBS=ON \
    -DBUILD_TESTING=OFF \
    -DHDF5_BUILD_EXAMPLES=OFF \
    -DHDF5_BUILD_CPP_LIB=ON \
    -DHDF5_BUILD_HL_LIB=ON

cmake --build ~/hdf5-build -j
cmake --install ~/hdf5-build
```

Configure the HDF5 environment variables:

```
export HDF5_ROOT="${HOME}/hdf5"
export LD_LIBRARY_PATH="${HDF5_ROOT}/lib:${LD_LIBRARY_PATH:-}"
```

Verify the installation:

```
ls "${HDF5_ROOT}/include/hdf5.h"
ls "${HDF5_ROOT}"/lib/libhdf5*
```

---

(3) HighFive Library (Required)

HighFive is a header-only C++ interface to HDF5. It does not need to be compiled, but its header files must be available when building the search-planning code.

Clone HighFive into your home directory:

```
cd ~
git clone --depth 1 --recursive \
    https://github.com/highfive-devs/highfive.git HighFive
```

Configure the HighFive path:

```
export HIGHFIVE_ROOT="${HOME}/HighFive"
```

Verify that the required headers are present:

```
ls "${HIGHFIVE_ROOT}/include/highfive/H5File.hpp"
```

HighFive is header-only, no separate build or installation step is required.

---

(4) Build the C++ Executables

Once all dependencies are installed, build the C++ project with:

```
cd ~/GRB-Search-Pipeline/search-planning

export HDF5_ROOT="${HOME}/hdf5"
export HIGHFIVE_ROOT="${HOME}/HighFive"
export LEMON_SOURCE_DIR="${HOME}/lemon-1.3.1"
export LEMON_BUILD_DIR="${LEMON_SOURCE_DIR}/build"
export LD_LIBRARY_PATH="${HDF5_ROOT}/lib:${LD_LIBRARY_PATH:-}"

cmake -S . -B build
cmake --build build -j
```

This produces two binaries in `build/`. Both are compiled from `src/main.cpp` and share the same search-planning implementation. CMake selects a different entry point for each binary:

- **`sp_train`** — compiled with `BUILD_SP_TRAIN` and dispatches to `main_train()`. It processes training maps and produces search-outcome training data.
- **`sp_verify`** — compiled with `BUILD_SP_VERIFY` and dispatches to `main_verify()`. It processes

evaluation maps and records whether the simulated search reaches the source before the deadline.

The Python drivers pass the destination CSV as the final command-line argument. The executable interfaces are:

```
sp_train <map> <tiling> <source_tile> <w_max> <w_acc> <dwell_time> <is_deepslow>
<output_csv>
sp_verify <map> <tiling> <source_tile> <w_max> <w_acc> <remaining_budget> <is_deepslow>
<output_csv>
```

Each invocation appends one result row directly to `<output_csv>`. The Python driver creates the parent result directory and selects the final filename.

Verify that both binaries were created:

```
ls build/sp_train build/sp_verify
```

1.2 Download the Experiment Data

The detector models and simulated transient datasets are distributed separately from the source-code repository because of their size. The compressed archive is approximately 16.2 GB; allow approximately 25–30 GB of storage after extraction.

The data archive is available from Zenodo:

- **Zenodo record:** https://zenodo.org/records/21497891
- **Archive:** `transients.tar.gz`
- **Download size:** 16.2 GB
- **MD5:** `ba6f9511b99b5fb3faaebfaae01e03ac`

The archive contains the complete `transients/` directory and should be extracted directly under the user's home directory.

1.2.1 Download the Archive

Download the archive using a browser from the Zenodo record above, or run:

```
export TRANSIENTS_URL="https://zenodo.org/records/21497891/files/transients.tar.gz?
download=1"
export TRANSIENTS_ARCHIVE="${HOME}/transients.tar.gz"

curl \
    --location \
    --fail \
    --continue-at - \
    --output "${TRANSIENTS_ARCHIVE}" \
    "${TRANSIENTS_URL}"
```

The `--continue-at -` option allows an interrupted download to resume when supported by the server.

Alternatively, use `wget`:

```
wget -c \
    "https://zenodo.org/records/21497891/files/transients.tar.gz?download=1" \
    -O "${HOME}/transients.tar.gz"
```

1.2.2 Verify the Download

Zenodo publishes the MD5 checksum for the archive. Verify it before extracting:

```
cd "${HOME}"

echo "ba6f9511b99b5fb3faaebfaae01e03ac  transients.tar.gz" \
    | md5sum --check
```

The expected output is:

```
transients.tar.gz: OK
```

If the checksum does not match, remove the incomplete archive and download it again.

1.2.3 Extract the Dataset

Extract the archive under the home directory:

```
tar -xzf "${HOME}/transients.tar.gz" -C "${HOME}"
```

This should create:

```
~/transients/
├── models/
│   ├── adapt_response_w_area.h5
│   └── adapt_bkg_model.h5
├── emsoft_training_short/
├── emsoft_training_longlow/
├── emsoft_test_short/
├── emsoft_test_longlow/
└── dc3_benchmark_data/
```

Verify the extracted directories:

```
ls "${HOME}/transients"
ls "${HOME}/transients/models"
```

After successful extraction and verification, the downloaded archive may be removed to recover disk space:

```
rm "${HOME}/transients.tar.gz"
```

---

2 Reproducing Paper Figures

The distributed results used to produce the paper's figures are stored in `~/GRB-Search-Pipeline/precomputed_results/`. You can inspect and visualize these results with the provided Jupyter notebook without rerunning the experiments.

The `precomputed_results/` directory should remain unchanged. Results and figures generated by your own runs are written to `~/GRB-Search-Pipeline/results/`.

```
conda activate cosipy-312
cd ~/GRB-Search-Pipeline/precomputed_results

GRB_RESULTS_ROOT="${HOME}/GRB-Search-Pipeline/precomputed_results" \
GRB_FIGURE_OUTPUT_DIR="${HOME}/GRB-Search-Pipeline/results/figures" \
jupyter notebook visualize_Results.ipynb
```

The environment variables above make the notebook read the distributed results from `~/GRB-Search-Pipeline/precomputed_results/` and save newly generated figures under `~/GRB-Search-Pipeline/results/figures/`. Do not modify the distributed precomputed results.

The notebook should reproduce Figures 4-11 from the paper's results section. Figures 1-3 are explanatory diagrams rather than outputs of the experiment workflow.

3 Running Full Experiments

The full experiment has two phases:

1. **Training:** generate training maps, measure mapping and planning times, and collect the data used by the utility estimator.
2. **Evaluation:** generate maps for 10,000 test transients, run search planning, simulate the optical search, and calculate the success probability.

The full experiment is computationally expensive. To reproduce only the paper figures using the distributed results, skip this section and follow Section 2. The full experiment workflow below runs both Phase A and Phase B.

3.1 Phase A: Generate the Offline Training Data

The paper uses 52,800 training transients for each transient scenario. The training command processes every transient in the specified directory.

Step 1: Configure Paths

Activate the Python environment:

```
conda activate cosipy-312
```

Before starting any map-generation Python process, set the numerical-library thread limits used for the paper experiments. Here, we assume that NumPy was built to use the OpenBLAS library for multithreaded linear algebra, rather than the alternative OpenMP or MKL libraries.

```
export OPENBLAS_NUM_THREADS=8
export MKL_NUM_THREADS=1
export OMP_NUM_THREADS=1
```

`OPENBLAS_NUM_THREADS` controls NumPy matrix operations and is distinct from the `-t 8` option, which controls the map-generation threads used directly by `map_adapt_transients.py`.

Configure the repository, input-data, model, and output paths:

```
export REPO_ROOT="${HOME}/GRB-Search-Pipeline"
export DATA_ROOT="${HOME}/transients"

export MAP_SCRIPT="${REPO_ROOT}/cosipy/cosipy/ts_map/map_adapt_transients.py"

export RESPONSE_MODEL="${DATA_ROOT}/models/adapt_response_w_area.h5"
export BACKGROUND_MODEL="${DATA_ROOT}/models/adapt_bkg_model.h5"

export SHORT_TRAINING_TRANSIENTS="${DATA_ROOT}/emsoft_training_short"
export LONGLOW_TRAINING_TRANSIENTS="${DATA_ROOT}/emsoft_training_longlow"

export SHORT_TEST_TRANSIENTS="${DATA_ROOT}/emsoft_test_short"
export LONGLOW_TEST_TRANSIENTS="${DATA_ROOT}/emsoft_test_longlow"

export TRAINING_ROOT="${REPO_ROOT}/results/training"
export TRAINING_MAP_ROOT="${TRAINING_ROOT}/maps"
export TRAINING_LOG_ROOT="${TRAINING_ROOT}/logs"

mkdir -p \
    "${TRAINING_MAP_ROOT}/short" \
    "${TRAINING_MAP_ROOT}/longlow" \
    "${TRAINING_LOG_ROOT}"
```

Step 2: Generate the Short-Transient Training Maps (~ 4 hours)

Run:

```
env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
python "${MAP_SCRIPT}" \
    --response "${RESPONSE_MODEL}" \
    --bkg-model "${BACKGROUND_MODEL}" \
    -t 8 \
    -n 64 \
    "${SHORT_TRAINING_TRANSIENTS}" \
    gt \
    -m \
    -o "${TRAINING_MAP_ROOT}/short" \
    > "${TRAINING_ROOT}/short_mapping_time_stats.csv" \
    2> "${TRAINING_LOG_ROOT}/short_mapping.log"
```

The command processes all 52,800 short training transients. Do not use `-s 10000` during training.
If errors encountered, please go to `~/GRB-Search-Pipeline/results/training/logs/*.log` file for details.

The options specify:

- `-t 8`: use eight map-generation threads;
- `-n 64`: generate maps with HEALPix `nside = 64`;
- `gt`: use the true transient duration as the event-gathering endpoint;
- `-m`: write each generated map to disk; and
- `-o`: select the map output directory.

The generated mapping statistics are written to:

```
~/GRB-Search-Pipeline/results/training/short_mapping_time_stats.csv
```

Step 3: Generate the Long-Transient Training Maps (~ 4 hours)

Run:

```
env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
python "${MAP_SCRIPT}" \
    --response "${RESPONSE_MODEL}" \
    --bkg-model "${BACKGROUND_MODEL}" \
    -t 8 \
    -n 64 \
    "${LONGLOW_TRAINING_TRANSIENTS}" \
    gt \
    -m \
    -o "${TRAINING_MAP_ROOT}/longlow" \
    > "${TRAINING_ROOT}/longlow_mapping_time_stats.csv" \
    2> "${TRAINING_LOG_ROOT}/longlow_mapping.log"
```

The likelihood maps are independent of the optical telescope FoV. Therefore, each training transient is mapped only once.

Step 4: Run GCP on the Training Maps (~ 10 hours)

Run GCP on every generated training map for the following four scenario/FoV combinations:

- short transients with the `2.5 x 2.5` FoV;
- short transients with the `5.36 x 4.5` FoV;
- longlow transients with the `2.5 x 2.5` FoV; and
- longlow transients with the `5.36 x 4.5` FoV.

The required tiling and true-source lookup files are pre-generated:

```
search-planning/tilings/
├── tiling_files/
│   ├── 2.5x2.5_tiling.csv
│   └── 5.36x4.5_tiling.csv
└── source_tile_train/
    ├── source_tiles_2.5x2.5_short.csv
    ├── source_tiles_5.36x4.5_short.csv
    ├── source_tiles_2.5x2.5_longlow.csv
    └── source_tiles_5.36x4.5_longlow.csv
```

Open `run_training.py` and configure:

```
FOVS = ["2.5x2.5", "5.36x4.5"]

TRAINING_MAP_DIRS = {
    "short": REPO_ROOT / "results" / "training" / "maps" / "short",
    "longlow": REPO_ROOT / "results" / "training" / "maps" / "longlow",
}

EXECUTABLE = SEARCH_ROOT / "build" / "sp_train"
RESULTS_DIR = REPO_ROOT / "results" / "training"
```

The generated maps use the Bitshuffle HDF5 compression filter. Before running the C++ program, make the corresponding plugin available to HDF5:

```
conda activate cosipy-312

export HDF5_PLUGIN_PATH="$(
    python -c "import hdf5plugin; print(hdf5plugin.PLUGIN_PATH)"
)"
export LD_LIBRARY_PATH="${HOME}/hdf5/lib:${LD_LIBRARY_PATH:-}"
```

Go to the directory containing `run_training.py`, run:

```
cd ~/GRB-Search-Pipeline/results
python run_training.py
```

The script invokes `sp_train` on every training map for each configured FoV. For each map, GCP records the planning runtime and the minimum search budget needed to reach the tile containing the true source. The script passes the corresponding final training CSV path directly to `sp_train`.

The four output files are:

```
~/GRB-Search-Pipeline/results/training/
├── short_searching_2.5x2.5_tiling.csv
├── short_searching_5.36x4.5_tiling.csv
├── longlow_searching_2.5x2.5_tiling.csv
└── longlow_searching_5.36x4.5_tiling.csv
```

The mapping statistics and GCP outcomes have different roles:

- `short_mapping_time_stats.csv` contains measurements used to predict map-generation time.
- `short_searching_5.36x4.5_tiling.csv` contains outcomes used to estimate the probability of reaching the source within a given search budget.

The corresponding longlow and `2.5x2.5` files serve the same purposes.

3.2 Phase B: Evaluate the Policies

The evaluation uses 10,000 test transients from each scenario. The fixed random seed `1957` selects the same test transients for every policy and deadline.

Step 5: Configure the Evaluation Inputs

Configure the evaluation output directories:

```
export VALIDATION_ROOT="${REPO_ROOT}/results/validation"
export VALIDATION_MAP_ROOT="${VALIDATION_ROOT}/maps"
export VALIDATION_STATS_ROOT="${VALIDATION_ROOT}/mapping_stats"
export VALIDATION_LOG_ROOT="${VALIDATION_ROOT}/logs"

mkdir -p \
    "${VALIDATION_MAP_ROOT}" \
    "${VALIDATION_STATS_ROOT}" \
    "${VALIDATION_LOG_ROOT}"
```

Use the training data generated in Phase A:

```
export SHORT_MAPPING_TIMES="${TRAINING_ROOT}/short_mapping_time_stats.csv"
export LONGLOW_MAPPING_TIMES="${TRAINING_ROOT}/longlow_mapping_time_stats.csv"
export TRAINING_OUTCOMES_ROOT="${TRAINING_ROOT}"
```

The validation output uses the same scenario/FoV directory names as `precomputed_results/validation/`, allowing the same visualization notebook to read either result tree.

Step 6: Generate Utility-Guided Test Maps (~ 10 hours)

Utility-guided mapping depends on the telescope FoV because each FoV uses a different GCP training-outcome table. Generate a separate set of test maps for every FoV and deadline.

For short transients, run:

```
for FOV in 2.5x2.5 5.36x4.5; do
    for DEADLINE in 10 15 20 25 30 35 40 45 50; do
        CASE_NAME="${FOV}_short"
        MAP_CASE_ROOT="${VALIDATION_MAP_ROOT}/${CASE_NAME}"
        STATS_CASE_ROOT="${VALIDATION_STATS_ROOT}/${CASE_NAME}"
        LOG_CASE_ROOT="${VALIDATION_LOG_ROOT}/${CASE_NAME}"

        mkdir -p \
            "${MAP_CASE_ROOT}" \
            "${STATS_CASE_ROOT}" \
            "${LOG_CASE_ROOT}"

        echo "Running short: FoV=${FOV}, deadline=${DEADLINE}"

        env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
        python "${MAP_SCRIPT}" \
            --response "${RESPONSE_MODEL}" \
            --bkg-model "${BACKGROUND_MODEL}" \
            -t 8 \
            -n 64 \
            -s 10000 \
            -r 1957 \
            --training-times "${SHORT_MAPPING_TIMES}" \
            --training-outcomes \
                "${TRAINING_OUTCOMES_ROOT}/short_searching_${FOV}_tiling.csv" \
            "${SHORT_TEST_TRANSIENTS}" \
            "${DEADLINE}" \
            -m \
            -o "${MAP_CASE_ROOT}/emsoft_maps_${DEADLINE}" \
            > "${STATS_CASE_ROOT}/emsoft_stats_${DEADLINE}.csv" \
            2> "${LOG_CASE_ROOT}/emsoft_maps_${DEADLINE}.log"
    done
done
```

For longlow transients, run:

```
for FOV in 2.5x2.5 5.36x4.5; do
    for DEADLINE in 30 40 50 60 70 80 90 100 110; do
        CASE_NAME="${FOV}_longlow"
        MAP_CASE_ROOT="${VALIDATION_MAP_ROOT}/${CASE_NAME}"
        STATS_CASE_ROOT="${VALIDATION_STATS_ROOT}/${CASE_NAME}"
        LOG_CASE_ROOT="${VALIDATION_LOG_ROOT}/${CASE_NAME}"

        mkdir -p \
            "${MAP_CASE_ROOT}" \
            "${STATS_CASE_ROOT}" \
```

```
            "${LOG_CASE_ROOT}"

        echo "Running longlow: FoV=${FOV}, deadline=${DEADLINE}"

        env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
        python "${MAP_SCRIPT}" \
            --response "${RESPONSE_MODEL}" \
            --bkg-model "${BACKGROUND_MODEL}" \
            -t 8 \
            -n 64 \
            -s 10000 \
            -r 1957 \
            --training-times "${LONGLOW_MAPPING_TIMES}" \
            --training-outcomes \
                "${TRAINING_OUTCOMES_ROOT}/longlow_searching_${FOV}_tiling.csv" \
            "${LONGLOW_TEST_TRANSIENTS}" \
            "${DEADLINE}" \
            -m \
            -o "${MAP_CASE_ROOT}/emsoft_maps_${DEADLINE}" \
            > "${STATS_CASE_ROOT}/emsoft_stats_${DEADLINE}.csv" \
            2> "${LOG_CASE_ROOT}/emsoft_maps_${DEADLINE}.log"
    done
done
```

The `-s 10000` option selects 10,000 test transients. The explicit `-r 1957` option ensures that every run uses the same test subset.
If errors are encountered, please go to corresponding `*.log` file for details.

Step 7: Generate Deadline-Oblivious Test Maps (~ 1 hours)

The `nodeadline` endpoint mode does not use the utility training tables and does not depend on FoV. Generate it once per transient scenario, then reuse the resulting maps for both FoVs and every deadline.

For short transients, run:

```
CASE_NAME="no-fov_short"
MAP_CASE_ROOT="${VALIDATION_MAP_ROOT}/${CASE_NAME}"
STATS_CASE_ROOT="${VALIDATION_STATS_ROOT}/${CASE_NAME}"
LOG_CASE_ROOT="${VALIDATION_LOG_ROOT}/${CASE_NAME}"

mkdir -p \
    "${MAP_CASE_ROOT}" \
    "${STATS_CASE_ROOT}" \
    "${LOG_CASE_ROOT}"

env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
python "${MAP_SCRIPT}" \
    --response "${RESPONSE_MODEL}" \
    --bkg-model "${BACKGROUND_MODEL}" \
    -t 8 \
    -n 64 \
    -s 10000 \
```

```
    -r 1957 \
    "${SHORT_TEST_TRANSIENTS}" \
    nodeadline \
    -m \
    -o "${MAP_CASE_ROOT}/emsoft_maps_nodeadline" \
    > "${STATS_CASE_ROOT}/emsoft_stats_nodeadline.csv" \
    2> "${LOG_CASE_ROOT}/emsoft_maps_nodeadline.log"
```

For longlow transients, run:

```
CASE_NAME="no-fov_longlow"
MAP_CASE_ROOT="${VALIDATION_MAP_ROOT}/${CASE_NAME}"
STATS_CASE_ROOT="${VALIDATION_STATS_ROOT}/${CASE_NAME}"
LOG_CASE_ROOT="${VALIDATION_LOG_ROOT}/${CASE_NAME}"

mkdir -p \
    "${MAP_CASE_ROOT}" \
    "${STATS_CASE_ROOT}" \
    "${LOG_CASE_ROOT}"

env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
python "${MAP_SCRIPT}" \
    --response "${RESPONSE_MODEL}" \
    --bkg-model "${BACKGROUND_MODEL}" \
    -t 8 \
    -n 64 \
    -s 10000 \
    -r 1957 \
    "${LONGLOW_TEST_TRANSIENTS}" \
    nodeadline \
    -m \
    -o "${MAP_CASE_ROOT}/emsoft_maps_nodeadline" \
    > "${STATS_CASE_ROOT}/emsoft_stats_nodeadline.csv" \
    2> "${LOG_CASE_ROOT}/emsoft_maps_nodeadline.log"
```

After Steps 6 and 7, the generated mapping statistics are organized as:

```
results/validation/mapping_stats/
├── 2.5x2.5_longlow/
├── 2.5x2.5_short/
├── 5.36x4.5_longlow/
├── 5.36x4.5_short/
├── no-fov_longlow/
└── no-fov_short/
```

The `maps/` and `logs/` directories use the same six case names. This matches the case naming under `precomputed_results/validation/mapping_stats/`.

Step 8: Run GCP and Simulate the Search (~ 10 hours)

The `run_validation.py` script evaluates the generated test maps using the `sp_verify` executable. It passes the appropriate FoV-specific result CSV path directly to each `sp_verify` invocation.

The validation source-tile lookup files must be stored under:

```
search-planning/tilings/source_tile_validation/
├── source_tiles_2.5x2.5_short.csv
├── source_tiles_5.36x4.5_short.csv
├── source_tiles_2.5x2.5_longlow.csv
└── source_tiles_5.36x4.5_longlow.csv
```

Open `run_validation.py` and configure:
(The default configuration exactly produces the results in the paper, keep them as they are if you don't want to make any change.)

```
SCENARIOS = ["short", "longlow"]
POLICIES = ["utility", "nodeadline"]
FOVS = ["2.5x2.5", "5.36x4.5"]

DEADLINES = {
    "short": [10, 15, 20, 25, 30, 35, 40, 45, 50],
    "longlow": [30, 40, 50, 60, 70, 80, 90, 100, 110],
}

VALIDATION_ROOT = REPO_ROOT / "results" / "validation"
MAP_ROOT = VALIDATION_ROOT / "maps"
MAPPING_STATS_ROOT = VALIDATION_ROOT / "mapping_stats"

UTILITY_MAP_ROOTS = {
    "short": str(MAP_ROOT / "{fov}_short"),
    "longlow": str(MAP_ROOT / "{fov}_longlow"),
}

UTILITY_STATS_ROOTS = {
    "short": str(MAPPING_STATS_ROOT / "{fov}_short"),
    "longlow": str(MAPPING_STATS_ROOT / "{fov}_longlow"),
}

BASELINE_MAP_ROOTS = {
    "short": MAP_ROOT / "no-fov_short",
    "longlow": MAP_ROOT / "no-fov_longlow",
}

BASELINE_STATS_ROOTS = {
    "short": MAPPING_STATS_ROOT / "no-fov_short",
    "longlow": MAPPING_STATS_ROOT / "no-fov_longlow",
}

EXECUTABLE = SEARCH_ROOT / "build" / "sp_verify"
SEARCH_RESULTS_ROOT = VALIDATION_ROOT
```

Make the HDF5 library and Bitshuffle reader plugin available to the C++ executable:

```
conda activate cosipy-312

export HDF5_PLUGIN_PATH="$(
    python -c "import hdf5plugin; print(hdf5plugin.PLUGIN_PATH)"
)"
export LD_LIBRARY_PATH="${HOME}/hdf5/lib:${LD_LIBRARY_PATH:-}"
```

From the directory containing `run_validation.py`, run:

```
cd ~/GRB-Search-Pipeline/results
python run_validation.py
```

For each test map, the script:

1. reads the gathering and mapping times from the corresponding statistics CSV;
2. subtracts those times from the overall deadline;
3. runs GCP and measures its planning time;
4. accounts for the remaining optical-search budget;
5. simulates following the planned path using the telescope timing model; and
6. records whether the path reaches the true source tile before the deadline.

The result files are grouped by FoV under:

```
~/GRB-Search-Pipeline/results/validation/
├── searching_results_2.5x2.5/
└── searching_results_5.36x4.5/
```

Each result filename has the following form:

```
<scenario>_<fov>_tiling_<policy>_<deadline>.csv
```

Examples include:

```
searching_results_2.5x2.5/short_2.5x2.5_tiling_utility_10.csv
searching_results_5.36x4.5/short_5.36x4.5_tiling_nodeadline_30.csv
searching_results_2.5x2.5/longlow_2.5x2.5_tiling_nodeadline_60.csv
searching_results_5.36x4.5/longlow_5.36x4.5_tiling_utility_110.csv
```

Step 9: Aggregate and Visualize the Recomputed Results (~ 10 minutes)

```
conda activate cosipy-312
cd "${REPO_ROOT}/results"
jupyter notebook visualize_Results.ipynb
```

The notebook reads the recomputed training and validation CSV files under:

```
~/GRB-Search-Pipeline/results/
```

and writes the generated figures under:

```
~/GRB-Search-Pipeline/results/figures/
```

4 Running the Mapping-Performance Benchmark (Optional)

This optional benchmark reproduces the mapping-time measurements in Figure 4. The complete figure contains five measurements on a 2.3 GHz Intel Xeon Gold 5118 system and three measurements on the Jetson Orin NX. Run the commands for each platform whose results you want to reproduce.

The five benchmark modes correspond to Figure 4 as follows:

Benchmark mode	Figure 4 label	Intel	Jetson
`cosipy_interp`	`cosipy v3`	yes	—
`cosipy_numba`	`cosipy + JIT`	yes	—
`emsoft_outmem`	`ext-mem`	yes	yes
`emsoft_inmem`	`in-mem`	yes	yes
`emsoft_moc`	`in-mem-mr`	yes	yes

The two original COSIpy implementations require more memory than is available on the 16 GB Jetson and therefore appear only in the Intel results.

4.1 Intel x86-64 Environment Setup

Use this environment only for the Intel/x86-64 benchmark. Verify that the machine reports `x86_64`:

```
uname -m
```

If Conda is not installed, install the x86-64 version of Miniconda:

```
cd ~/GRB-Search-Pipeline

export CONDA_DIR="${HOME}/conda-intel"

wget -q \
    https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh \
    -O miniconda-intel.sh

bash miniconda-intel.sh -b -p "${CONDA_DIR}"
rm miniconda-intel.sh
```

Create and activate the Intel Python environment:

```
. "${CONDA_DIR}/etc/profile.d/conda.sh"

cd ~/GRB-Search-Pipeline
conda env create -f cosipy-312-intel.yml
conda activate cosipy-312-intel
```

Install the artifact branch of COSIpy:

```
cd ~/GRB-Search-Pipeline/cosipy
git switch tsmap-artifact
git pull --ff-only

python -m pip install "poetry-core>=2,<3"
python -m pip install --no-deps -e .
```

4.2 Run the Intel Benchmarks (~ 20 minutes)

The benchmark inputs are provided in the downloaded data archive at:

```
~/transients/dc3_benchmark_data/
```

Configure the paths and common thread settings. These variables must be set before Python starts:

```
conda activate cosipy-312-intel
cd ~/GRB-Search-Pipeline/cosipy/cosipy/ts_map

export DC3_BENCHMARK_DATA="${HOME}/transients/dc3_benchmark_data"
export BENCHMARK_RESULTS="${HOME}/GRB-Search-Pipeline/results/dc3_benchmark/intel"
mkdir -p "${BENCHMARK_RESULTS}"
```

Run the two original COSIpy modes with one OpenBLAS thread. (The COSIpy modes use Python multiprocessing to start eight separate processes, so they still use eight processors, with one OpenBLAS thread in each):

```
export OPENBLAS_NUM_THREADS=1

for MODE in cosipy_interp cosipy_numba; do
    echo "Running ${MODE}"
    python dc3_benchmark.py \
        -d "${DC3_BENCHMARK_DATA}" \
        "${MODE}" \
        | tee "${BENCHMARK_RESULTS}/${MODE}.txt"
done
```

Run the three EMSOFT modes with eight OpenBLAS threads. (The EMSOFT modes use a single process, so the number of processors used remains the same):

```
export OPENBLAS_NUM_THREADS=8

for MODE in emsoft_outmem emsoft_inmem emsoft_moc; do
    echo "Running ${MODE}"
    python dc3_benchmark.py \
        -d "${DC3_BENCHMARK_DATA}" \
        "${MODE}" \
        | tee "${BENCHMARK_RESULTS}/${MODE}.txt"
done
```

4.3 Run the Jetson Benchmarks (~ 10 minutes)

Use the ARM64 environment configured in Section 1.1.2, then run the three EMSOFT modes.

```
conda activate cosipy-312
cd ~/GRB-Search-Pipeline/cosipy/cosipy/ts_map

export DC3_BENCHMARK_DATA="${HOME}/transients/dc3_benchmark_data"
export BENCHMARK_RESULTS="${HOME}/GRB-Search-Pipeline/results/dc3_benchmark/jetson"
mkdir -p "${BENCHMARK_RESULTS}"

export OPENBLAS_NUM_THREADS=8

for MODE in emsoft_outmem emsoft_inmem emsoft_moc; do
    echo "Running ${MODE}"
    python dc3_benchmark.py \
        -d "${DC3_BENCHMARK_DATA}" \
        "${MODE}" \
        | tee "${BENCHMARK_RESULTS}/${MODE}.txt"
done
```

The benchmark uses eight CPU cores. Each run reports the average of five measured iterations after one warm-up iteration:

```
TIME: 0.402 s
```

4.4 Visualize the Benchmark Results

Display the measured times recorded in the benchmark output files:

```
grep "TIME:" \
    ~/GRB-Search-Pipeline/results/dc3_benchmark/intel/*.txt

grep "TIME:" \
    ~/GRB-Search-Pipeline/results/dc3_benchmark/jetson/*.txt
```

Activate the environment appropriate for the current platform:

```
# On Intel:
conda activate cosipy-312-intel
```

Install Jupyter Notebook if it is not already available:

```
python -m pip install notebook
```

Launch the visualization notebook:

```
cd ~/GRB-Search-Pipeline/precomputed_results

GRB_RESULTS_ROOT="${HOME}/GRB-Search-Pipeline/precomputed_results" \
GRB_FIGURE_OUTPUT_DIR="${HOME}/GRB-Search-Pipeline/results/figures" \
jupyter notebook visualize_Results.ipynb
```

In the notebook, find **Optional: Reproduce Figure 4** and copy the numerical value from each `TIME:` line into the corresponding entry of `time_means`.

Each tuple in `time_means` is ordered as:

```
(Intel_time, Jetson_time)
```

All times are in seconds. Replace every `None` with the corresponding measured time. Keep `np.nan` for the two original COSIpy implementations that are not run on the Jetson.